

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3090 - Ingeniería de software 2

Sección 20

Ing. Pablo Koch



Tarea de mejora al proceso | Revisión Técnica Formal

Diego Patzán - 23525

Diego López - 23242

June Herrera - 231038

Ihan Marroquin – 23108

GUATEMALA, 01 de agosto de 2025

Planificación de la RTF

1. Número de Inspección:

_____01_____

Fecha: 12/08/2025

2. Equipo de trabajo

Ingeniero 1

Diego López

Ingeniero 2

Diego Patzán

Ingeniero 3

Ihan Marroquín

Ingeniero 4

June Herrera

3. Documentos

Producto a Revisar

Corporación de Servicios Profesionales e Inmobiliarios

Checklist

[Lista de cotejo](#)

4. Reuniones

Planificación

12 de agosto del año 2025

Inspección

12 de agosto del año 2025

5. Objetivos de Planificación

- Se tiene documentación de los productos a revisar
- Se tienen checklists de los productos a revisar
- Los miembros del equipo están de acuerdo en las fechas propuestas para reuniones.
- La revisión rápida del Moderador produce menos de 5 defectos importantes.
- Los inspectores entienden sus responsabilidades y se comprometen a la labor.

6. Tiempo planificación

· 120 minutos

Inspección de productos de software - Diego Lopez

(Lista de inspección)

1. Número de Inspección: 1 Fecha: 12/08/2025

2. Nombre inspector: Diego López

3. Documentos LoginScreen.kt,
[SPagoEnviarViewModel.kt](#),
[SPagoEnviarRepository.kt](#),
[SPagoStatusScreen.kt](#),
 DPaymentScreen.kt,
 DPaymentsViewModel.kt etc.

4. Defectos (Criticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	LoginScreen.kt (solo texto "¿Olvidaste tu contraseña?" sin acción); debería tener flujo y endpoint en el ViewModel y Repository	Severos	Falta flujo funcional y backend para recuperación.	Implementar pantalla o modal con formulario para email, endpoint en LoginRepository, y manejo de estado en LoginViewModel.
2	SPagoEnviarViewModel.kt + SPagoEnviarRepository.kt (usa Mock, falta integración backend)	Críticos	MockData y lógica de presentación, pero falta persistencia real.	Integrar API/backend y persistencia real en el repositorio y ViewModel.
3	Esperado en: SPagoStatusScreen.kt y SPagosStatusRepository.kt (MockData,	Severos	Solo mock, falta consulta real y visualización de comprobante.	Integrar consulta a backend y agregar visualización de estado y comprobante (imagen/bool) en la pantalla.

	falta consulta real)			
4	No implementado, esperado en lógica de SPagoEnviar ViewModel.kt o servicio de notificaciones	Moderada	No hay lógica de notificación ni integración con backend/servicio push.	Implementar lógica de notificaciones (push/email), preferentemente como servicio, y disparador por fecha.
5	En SPagoEnviar ViewModel.kt y MockData, pero solo permite agregar uno por flujo	Severos	Falta soporte en la lógica y estructura de datos para múltiples comprobantes.	Adaptar los modelos y la UI para permitir agregar varios comprobantes por mes y ajustar la persistencia.
6	Parcialmente implementado (SPagoEnviar ViewModel.kt y SPagosEnviar Screen.kt permiten agregar), pero solo Mock	Moderada	Solo mock, falta integración real con backend.	Agregar integración real y persistencia en backend para aportes a capital en pagos.
7	DPaymentScreen.kt, DPaymentsViewModel.kt (usa MockData)	Severos	Solo MockData, falta persistencia y consulta real.	Integrar con backend y mostrar datos reales en la pantalla de pagos registrados.
8	DPendingPay ViewModel.kt, DPendingPay Screen.kt; métodos onApprove/onReject con TODO	Críticos	Métodos de validación/rechazo o marcados como TODO, falta lógica y llamada backend.	Implementar lógica de validación/rechazo y retroalimentación, con integración a backend y manejo de estado.
9	DFineManagerViewModel.kt, DFineManagerRepository.kt (submitFine con TODO)	Críticos	Método submitFine sin implementación, falta integración y persistencia.	Implementar submitFine, lógica de UI y persistencia en backend, actualizar listado de multas.

5. Tiempo de preparación: 50 min

6. Total de defectos: críticos 3 severos 4 Moderados 2 Menores 0 preguntas 2

Inspección de productos de software - Diego Patzan

1. Número de Inspección: 2 Fecha: 12/08/2025

2. Nombre inspector: Diego Patzan 3. Documentos

4. Defectos (Críticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	Autenticación (Frontend/API)	Critico	Recuperación segura de contraseñas	Agregar mecanismo de recuperación con enlace temporal y validación de expiración.
2	Pagos (Frontend/API)	Severo	Registro de pagos	Completar integración y validación para guardar pagos correctamente.
3	Pagos (Frontend)	Severo	Estado de comprobante de pago	Crear vista donde los socios puedan consultar el estado de sus comprobantes.
4	Notificaciones (Frontend/API)	Severo	Aviso de pagos pendientes	Añadir sistema automático de alertas antes del vencimiento mensual.

5	Pagos (Frontend/API)	Severo	Comprobantes múltiples por mes	Permitir registro y visualización de varios comprobantes en un mismo periodo.
6	Pagos (Frontend)	Severo	Aportes a capital	Facilitar opción de aporte a capital al registrar pagos; reflejarlo en históricos.
7	Gestión (Directiva)	Severo	Visualización y validación de pagos	Implementar interfaz para que la directiva revise y valide pagos con retroalimentación.
8	Sanciones (Directiva)	Severo	Asignación de multas	Desarrollar formulario y lógica para asignar y notificar sanciones/multas.
9	Consultas (Directiva/Frontend)	Critico	Estado de socios y recuperación de contraseña	Mejorar consulta de estado de pagos y añadir recuperación de contraseña con caducidad.

5. Tiempo de preparación: 45 min

6. Total de defectos: criticos 2 severos 7 Moderados 0 Menores 0 preguntas 0

-

Inspección de productos de software - Ihan Marroquin

1. Número de Inspección: 3

Fecha: 12/08/2025

2. Nombre inspector: Ihan Marroquin

3. Documentos ListaCotejoCSP.xlsx,
Repositorios:
Bamo0507/Cooperativa_Frontend
dlop6/directiva-handler
Bamo0507/general-api-handler-cooperativa

4. Defectos (Críticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	Cooperativa_Frontend / general-api-handler-cooperativa	Crítico	No se logra recuperar la contraseña	No implementado: Restablecer contraseña en caso de olvido.
2	Cooperativa_Frontend	Moderado	Aún no realiza el pago.	No implementado: Registrar envío de pago en BD. Cerca de cumplirlo.
3	Cooperativa_Frontend	Moderado	Aún falta el poder ver el comprobante del pago	No implementado: Visualizar estado de comprobante de pago.
4	Cooperativa_Frontend	Moderado	No realiza la notificación.	No implementado: Notificar pagos pendientes antes del 5 de cada mes.
5	Cooperativa_Frontend	Moderado	No permite aun pagos múltiples	No implementado: Permitir múltiples comprobantes de pago por mes.
6	Cooperativa_Frontend	Moderado	No existe aporte a capital	No implementado: Permitir aportes a capital en pagos.
7	directiva-handler	Moderado	No existe visualización de pagos	No implementado: Visualizar pagos registrados por la directiva.
8	directiva-handler	Moderado	Aún no existe la asignación a socios	No implementado: Asignar multas a socios.
9	Cooperativa_Frontend / general-api-handler-cooperativa	Crítico	Aún no existe la recuperación de la contraseña	No implementado: Recuperación segura de contraseñas (enlace con caducidad).

5. Tiempo de preparación: 60 min

6. Total de defectos: críticos 2 severos 0 Moderados 7 Menores 0 preguntas 0

Inspección de productos de software - June Herrera

1. Número de Inspección: _____ 4 _____

Fecha: ____ 12/08/2025 _____

2. Nombre inspector: ____ June Herrera _____

3. Documentos LoginRepositoryRest.kt, LoginViewModel.kt, PreferencesDataStore.kt, PreferencesLocalStorage.kt, SPagoEnviarRepository.kt, DPaymentsRepository.kt, DPaymentsDetailRepository.kt, DPendingPayRepository.kt, SHistorialRepository.kt, DFinesRepository.kt, DPagaresRepository.kt

4. Defectos (Críticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	composeApp/src/commonMain/kotlin/app/cooperativa/domain/login/(LoginRepository*, LoginRepositoryRest.kt, LoginViewModel.kt)	Crítico	Requisito Funcional: Restablecer contraseña / Requisito No Funcional: Recuperación segura de contraseñas – No existe flujo (pantallas, endpoint, expiración de enlace)	Implementar pantalla de “Olvidé mi contraseña”, endpoint POST /auth/forgot + /auth/reset con token de un solo uso y expiración, no revelar si el usuario existe, forzar cambio de contraseña al usar el enlace.
2	composeApp/.../domain/login/LoginRepositoryRest.kt (líneas de request);	Crítico	Uso de HttpMethod.Get para login con cuerpo JSON + userType hard-codeado "directive"	Cambiar a POST /general/login, no enviar credenciales en GET (riesgo de caché / proxies); mapear correctamente user_type devuelto por backend; validar token no vacío y manejar expiración; añadir manejo centralizado de errores y reintentos exponenciales si aplica.

	LoginResult.kt			
3	composeApp/src/main/kotlin/app/cooperativa/domain/socios / y /directiva/ (repositorios Mock* - SPagoEnviarRepositorio, DPaymentsRepository, DPaymentsDetailRepository, DPendingPaymentRepository)	Crítico	Requisitos Funcionales de Pagos (Registrar envío de pago, Visualizar estado, Notificar pagos pendientes) – solo datos mock, falta integración real	Sustituir mocks por implementación real (REST/GraphQL) con capa de datasource remota, persistencia local (cache) y estados UI (loading/error). Añadir notificaciones programadas (WorkManager / background timer) para avisos antes del día 5.
4	composeApp/src/main/kotlin/app/cooperativa/domain/directiva/ (DPendingPaymentRepository, DPaymentsDetailRepository, DFinesRepository, DPagaresRepository) y Payment/Fine mocks	Severo	Requisitos Funcionales: Validar/Rechazar pagos, Asignar multas, Añadir préstamos – faltan operaciones mutacionales (solo lecturas mock)	Definir métodos para aprobar/rechazar (PUT/PATCH), crear multas y préstamos (POST). Agregar estados de resultado y rollback optimista en UI. Diseñar DTOs y actualizar ViewModels correspondientes.
5	composeApp/src/main/kotlin/app/coo	Severo	Requisito Funcional: Historial de pagos / Consultar estado	Implementar consulta real (GraphQL GetHistoryQuery ya parcialmente usado). Mapear pagos, préstamos, cuotas y multas en

	perativa/domain/socios/SHistorialRepository.kt		de pagos de socios – getPrestamosByUser devuelve lista vacía (TODO)	un modelo unificado HistoryResponse; agregar paginación y filtros por fecha/estado.
6	composeApp/src/main/kotlin/app/cooperativa/domain/socios/SPagoEnviarRepository.kt y lógica de pagos	Severo	Requisitos Funcionales: Múltiples comprobantes por mes, Aportes a capital, Visualizar pagos registrados – no hay soporte (mocks sin campos ni lógica de acumulación)	Extender modelo Payment para múltiples comprobantes (lista de evidencias), campo tipo (cuota/préstamo/aporte capital), acumuladores por socio. UI: formulario dinámico que permita añadir varios archivos y discriminar montos. Validaciones en cliente y servidor.
7	composeApp/src/main/kotlin/app/cooperativa/data/preferences/PreferencesDataStore.kt	Crítico	Almacenamiento de pass_code en texto plano (setPass_code / getPass_code)	Eliminar almacenamiento de la contraseña. Guardar solo token de acceso (y refresh si existe). Si se requiere “recordar usuario”, almacenar solo user_name. Borrar claves sensibles en logout. Considerar cifrado (EncryptedSharedPreferences / KMM Secure Storage).
8	Arquitectura general (no hay mediciones de rendimiento / límites de concurrencia visibles)	Severo	Requisitos No Funcionales: Soporte 30 usuarios simultáneos, transacciones en tiempo prudencial, consultas <3s – no hay métricas ni controles	Añadir capa de medición (logging, timestamps, métricas con Ktor plugin). Implementar caching local (Room/SQLDelight) para lecturas frecuentes. Optimizar llamadas paralelas con coroutine scopes estructuradas. Definir SLIs y pruebas de carga (k6 / Gatling).
9	composeApp/src/main/kotlin/app/cooperativa/domain/login/LoginRepository.kt (baseUrl),	Moderado	Configuración rígida (baseUrl hard-codeado), falta diferenciación de entornos; requisito de claridad y mantenibilidad	Externalizar baseUrl vía build config / inyección (expect/actual o gradle buildConfig). Añadir soporte a múltiples entornos (dev, staging, prod) y feature flags para activar módulos conforme se implementen.

	auth/koin modules			
--	----------------------	--	--	--

5. Tiempo de preparación: 70 min

6. Total de defectos: criticos severos Moderados Menores preguntas

Revisión Técnica Formal. Plantilla de reunión de inspección.

Cada uno de los revisores debe haber contabilizado cada uno de los aspectos listados en la tabla siguiente durante la inspección

Agregar datos de hojas de chequeo de cada inspector

	R1	R2	R3	R4	R5	R6	Total
Tiempo preparación	<u>50</u>	<u>45</u>	<u>60</u>	<u>70</u>	<u> </u>	<u> </u>	= <u>225</u> min
Defectos Criticos	<u>3</u>	<u>2</u>	<u>2</u>	<u>4</u>	<u> </u>	<u> </u>	= <u>11</u> iss.
Defectos Severos	<u>4</u>	<u>7</u>	<u>0</u>	<u>4</u>	<u> </u>	<u> </u>	= <u>15</u> iss.
Defectos Moderados	<u>2</u>	<u>0</u>	<u>7</u>	<u>1</u>	<u> </u>	<u> </u>	= <u>10</u> iss.
Defectos Menores	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u> </u>	<u> </u>	= <u>0</u> iss.
Preguntas al Autor	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u> </u>	<u> </u>	= <u>0</u> Qs

- Objetivos de la Reunión
- o Todo el equipo está presente. Listar ausentes: _____
 - o Todo el equipo se preparó adecuadamente para la reunión.
 - o Todos los aspectos registrados por Secretario son entendidos por Autor para su retrabajo.
 - o Se registraron los problemas que presenta el proceso de inspección.

Tiempo reunión Insp. 80 min. meet x 4 particip. = 320 min

Corporación de Servicios Profesionales e Inmobiliarios

Informe de Revisión técnica formal (RTF)

Versión [1.0]

Historia de revisiones

Fecha	Versión	Descripción	Autor
[12/08/2025]	[1.0]	Se revisaron funcionalidades generales, así como desperfectos visuales y cualquier otro tipo de comportamiento anormal que puede generar la UI para con el usuario	Diego Lopez Diego Patzán Ihan Marroquin June Herrera

Contenido

Corporación de Servicios Profesionales e Inmobiliarios 13

Informe de Revisión técnica formal (RTF) 13

Versión [1.0] 1

Historia de revisiones. 1

1. Producto revisado.. 4
 - 1.1. Nombre y Versión del Producto revisado. 4
 - 1.2. Participantes de la revisión. 4
 - 1.3. Técnica utilizada. 4
2. Objetivos de la RTF. 4
3. Problemas detectados. 4
 - 3.1. [Problema detectado 1] 4
 - 3.1.1. Sugerencia de corrección. 4
4. Evaluación. 4
 - 4.1. Estado actual del Producto. 4
 - 4.2. Acciones a tomar. 4
 - 4.3. Próxima Revisión del Producto. 5

1. Producto revisado

1.1. Nombre y Versión del Producto revisado

Corporación de Servicios Profesionales e Inmobiliarios

1.2. Participantes de la revisión

Diego López

Diego Patzán

Ihan Marroquín

June Herrera

1.3. Técnica utilizada

Se utilizó la Revisión Técnica Formal basada en una lista de cotejo definida como referencia en el Plan de SQA para esta revisión.

2. Objetivos de la RTF

- Verificar el cumplimiento de requisitos funcionales clave (autenticación, registro/consulta de pagos, notificaciones, gestión por la directiva).
- Evaluar aspectos no funcionales críticos: seguridad en almacenamiento de credenciales, rendimiento (soporte a usuarios simultáneos) y configuración/entornos.
- Detectar discrepancias entre diseño y código (uso de mocks donde debe haber integración real), y problemas de arquitectura o anti-patrón (e.g., envío de credenciales por GET).
- Revisar si las correcciones pendientes de versiones anteriores fueron implementadas (se constataron varios puntos pendientes).

3. Problemas detectados

3.1. Recuperación segura de contraseña (autenticación)

- Ubicación / archivos: Módulos de login: LoginScreen.kt, LoginRepositoryRest.kt, LoginViewModel.kt, repositorios generales.
- Severidad: Crítico (aparece repetido en varias hojas de inspección).
- Descripción: No existe flujo ni endpoint implementado para recuperación/restablecimiento de contraseña, ya que se detecta ausencia de pantalla/modal, ausencia de endpoints /auth/forgot y /auth/reset y ausencia de expiración/validación con token. Nuestra Checklist marca “Restablecer contraseña” como no cumplido.

3.1.1. Sugerencia de corrección

- Implementar pantalla “Olvidé mi contraseña” con formulario de correo.

3.2. Almacenamiento inseguro de credenciales

- Ubicación / archivos: PreferencesDataStore.kt, PreferencesLocalStorage.kt (uso de setPass_code / getPass_code).
- Severidad: Crítico.
- Descripción: Se ha detectado almacenamiento de pass_code (contraseña) en texto plano en preferencias locales; riesgo de exposición en caso de acceso al dispositivo.

3.2.1. Sugerencia de corrección

- Implementar almacenamiento seguro (EncryptedSharedPreferences, KeyStore o solución segura apropiada) y borrar credenciales al hacer logout.

3.3. Uso de MockData en los módulos de pagos / falta de integración backend

- Ubicación / archivos: SPagoEnviarViewModel.kt, SPagoEnviarRepository.kt, SPagoStatusScreen.kt, DPaymentScreen.kt, DPaymentsViewModel.kt, repositorios de pagos (múltiples).
- Severidad: Crítico / Severo.
- Descripción: Muchas pantallas y repositorios usan datos mock en vez de una datasource remota real, funcionalidades como registrar envío de pago, consulta de estado, agregar comprobantes, aprobar/rechazar pagos están incompletas.

3.3.1. Sugerencia de corrección

- Implementar capa de datasource remota (REST/GraphQL) y reemplazar mocks por llamadas reales.

3.4. Falta de funcionalidades de pagos (múltiples comprobantes, aportes a capital, visualización de comprobantes)

- Ubicación: Frontend pagos (pantallas y repositorios), nuestra Checklist marca múltiples requisitos de pago como “no cumplidos”.
- Severidad: Severo / Moderado.

- Descripción: No se permite registrar múltiples comprobantes por mes; no existe la función de aportes a capital; no se visualizan comprobantes ni su estado en el historial.

3.4.1. Sugerencia de corrección

- Extender el modelo Payment para soportar lista de evidencias (múltiples comprobantes), tipo de pago (cuota/préstamo/aporte), y metadatos (fecha subida, estado).
- UI: formulario dinámico para agregar múltiples archivos y asociarlos a la misma transacción.
- Backend: endpoints para recibir múltiples archivos y asociarlos; campos para diferenciar tipo de aporte.

3.5. Validación / acción de la Directiva (visualizar pagos, aprobar/rechazar, asignar multas, añadir préstamos)

- Ubicación: DPendingPayViewModel.kt, DFineManagerViewModel.kt, directiva-handler repositorio.
- Severidad: Severo / Crítico.
- Descripción: Las operaciones mutacionales (aprobar/rechazar, crear multa, registrar préstamo) están faltando; no se puede revisar ni validar pagos.

3.5.1. Sugerencia de corrección

- Implementar endpoints para aprobar/rechazar pagos, crear multas y préstamos.
- UI: vistas para confirmaciones, historia de acciones y retroalimentación al socio.

3.6. Lógica de notificaciones ausente (alertas de pagos pendientes)

- Ubicación: SPagoEnviarViewModel.kt y servicios de notificación (no implementado). Checklist marca “Notificar pagos pendientes” como no cumplido.
- Severidad: Moderada / Severo.
- Descripción: No existe lógica para enviar notificaciones push o email automáticas antes del vencimiento (por ejemplo antes del día 5).

3.6.1. Sugerencia de corrección

- Implementar servicio de notificaciones (Push + Email) con tareas programadas (WorkManager o servicio backend cron).

3.7. Falta de métricas, pruebas de carga y SLIs definidos (arquitectura / rendimiento)

- Ubicación: Arquitectura general (no hay mediciones visibles).
- Severidad: Severo.
- Descripción: Requisitos no funcionales como “soporte 30 usuarios simultáneos”, consultas <3s, etc., no están verificados con métricas ni pruebas de carga.

3.7.1. Sugerencia de corrección

- Realizar pruebas de carga (k6/Gatling), pruebas de stress, y optimizar cuello de botella (caching, paralelismo con coroutines).

4. Evaluación

4.1. Estado actual del Producto

Basado en la contabilización de defectos por los inspectores:

Defectos críticos: 11 issues.

Defectos severos: 15 issues.

Defectos moderados: 10 issues.

Defectos menores: 0.

Evaluación global: El producto se encuentra en una fase preliminar de desarrollo y no está listo para entrega productiva. Hay problemas críticos de seguridad (almacenamiento de credenciales, uso de GET para login), ausencia de flujo de recuperación de contraseña y falta de integración real para funcionalidades de pago y gestión por la directiva. Estas deficiencias impiden la aceptación del producto tal como está.

4.2. Acciones a tomar

Prioridad y acciones propuestas (propósito: corregir bloqueadores críticos y permitir nuevas pruebas):

Prioridad Alta (corrigir antes de siguiente revisión):

- Seguridad de credenciales: eliminar almacenamiento de pass_code en texto plano; implementar tokens y cifrado seguro.
- Recuperación de contraseña: implementar flujo UI + endpoints y token con expiración.

- Corregir método de login: cambiar GET→POST para login, eliminar hard-code de userType.
- Integración real de pagos: sustituir mocks por datasource remota e implementar endpoints para registrar pagos y subir comprobantes.

Prioridad Media:

- Notificaciones automáticas (recordatorios de pago).
- Funcionalidades para multiples comprobantes y aportes a capital.
- Implementar operaciones mutacionales para la Directiva (aprobar/rechazar, multas, préstamos).

Prioridad Baja / Arquitectura:

- Definir SLIs/SLAs y ejecutar pruebas de rendimiento / carga.

Para cada acción se recomienda asignar responsable(s) y definir historias/tickets con criterios de aceptación claros (incluyendo pruebas unitarias y de integración).

4.3. Próxima Revisión del Producto

Propuesta: realizar la próxima RTF una vez que se hayan cerrado los defectos críticos y se haya implementado la integración básica de pagos y la recuperación de contraseñas.

Recomendación práctica: programar la siguiente revisión después de completar el Sprint de correcciones críticas, idealmente 2 semanas después del 12/08/2025 (propuesta: 26/08/2025) o al final de la iteración donde se hayan resuelto las prioridades altas. En la próxima revisión se revisarán específicamente: seguridad de credenciales, flujo de recuperación de contraseña, login seguro, y evidencia de integración de pagos (endpoints + persistencia).

[LGP1]Se repite esto para cada problema detectado